LAPORAN PRAKTIKUM

Struktur Data

“ArrayList”

DISUSUN OLEH:

ABDUL JABBAR

2511533021

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM:

SHERLY SUKMADIRA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan penulisan materi ini yang membahas tentang *Stack* dalam struktur data. Penulisan ini disusun sebagai salah satu bentuk pemahaman dasar dalam mata kuliah pemrograman, khususnya dalam memahami konsep struktur data yang sangat penting dalam dunia informatika.

Stack merupakan salah satu struktur data yang bersifat *LIFO (Last In First Out)*, yaitu elemen yang terakhir masuk akan menjadi elemen pertama yang keluar. Konsep ini banyak digunakan dalam berbagai implementasi program seperti pengelolaan memori, proses undo-redo, serta evaluasi ekspresi dalam pemrograman. Dengan memahami Stack, mahasiswa diharapkan mampu mengaplikasikan konsep ini dalam menyelesaikan berbagai permasalahan komputasi secara lebih efektif.

Penulis menyadari bahwa dalam penyusunan materi ini masih terdapat kekurangan, baik dari segi penyajian maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi penyempurnaan di masa yang akan datang.

Akhir kata, semoga materi ini dapat memberikan manfaat dan menambah wawasan bagi para pembaca, khususnya mahasiswa di bidang informatika.

DAFTAR ISI

KATA PENGANTAR ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat 1
BAB II PEMBAHASAN 2
2.1 Praktikum Struktur data 2
2.1.1 Class stackArray 2
2.1.2 Class stackArrayDriver 3
2.1.3 Class StackPostfix 4
2.1.4 Class SiswaStack 6
2.1.5 Class NilaiMaksimum 8
BAB III PENDAHULUAN 9
3.1 Kesimpulan 9
3.2 Saran 9
DAFTAR PUSTAKA 9

BAB I
PENDAHULUAN

1.1 Latar Belakang

dalam dunia pemrograman, struktur data memiliki peran yang sangat penting dalam mengelola dan menyimpan data secara efisien. Salah satu struktur data dasar yang sering digunakan adalah *Stack*. Stack merupakan struktur data yang bekerja dengan prinsip *LIFO (Last In First Out)*, di mana data yang terakhir dimasukkan akan menjadi data pertama yang dikeluarkan. Konsep ini banyak diterapkan dalam berbagai sistem komputer, seperti pengelolaan memori, proses pemanggilan fungsi (call stack), serta fitur undo dan redo dalam aplikasi. Oleh karena itu, pemahaman tentang Stack menjadi hal yang penting bagi mahasiswa informatika agar mampu mengembangkan program yang lebih terstruktur dan efisien.

1.2 Tujuan

Adapun tujuan dari pembelajaran mengenai Stack adalah untuk memahami konsep dasar struktur data Stack beserta cara kerjanya, mengetahui operasi-operasi dasar dalam Stack seperti *push*, *pop*, dan *peek*, serta mampu mengimplementasikan Stack dalam bahasa pemrograman untuk menyelesaikan berbagai permasalahan. Selain itu, mahasiswa diharapkan dapat memahami penerapan Stack dalam kasus nyata di dunia pemrograman.

1.3 Manfaat

Manfaat yang diperoleh dari mempelajari Stack adalah meningkatkan pemahaman tentang struktur data yang bersifat dinamis, membantu dalam pengembangan logika berpikir yang sistematis, serta mempermudah dalam menyelesaikan masalah yang berkaitan dengan pengolahan data secara berurutan. Selain itu, pemahaman Stack juga sangat berguna dalam pengembangan aplikasi, seperti dalam pembuatan fitur navigasi, pengelolaan ekspresi matematika, dan berbagai algoritma lainnya.

BAB II
PEMBAHASAN

2.1 Praktikum Struktur data

Pada pekan 3 Mahasiswa informatika menyalin kode dan mempelajari stack dalam java

2.1.1 Class stackArray

```java
public class stackArray_2511533021 {
    static final int MAX_3021= 1000;
    int top_3021;
    int a[] = new int [MAX_3021];
    boolean isEmpty_3021()
    {
        return (top_3021 < 0) ;
    }
    stackArray_2511533021()
    {
        top_3021=-1;
    }
    boolean push_3021 (int x)
    {
        if (top_3021>= (MAX_3021 - 1)) {
            System.out.println("Stack Overflow");
            return false;
        }
        else {
            a[++top_3021] = x;
            System.out.println( x  + " dimasukkan dalam stack");
            return true;
        }
    }
    int pop_3021()
    {
        if (top_3021 <= 0 ) {
            System.out.println("stack underflow");
            return 0;
```

```java
        }else {
            int x = a[top_3021--];
            return x;
        }
    }
    int peek_3021()
    {
        if (top_3021<0)
        {
            System.out.println("stack underflow");
            return 0;
        }
        else
        {
            int x = a [top_3021];
            return x;
        }
    }
    void print_3021 () {
        for (int i=top_3021; i>-1;i--) {
            System.out.print(" " +a[i]);
        }
    }

}
```

1. static final int MAX_3021 = 1000; int top_3021; int a[] = new int[MAX_3021];

Mendeklarasikan kapasitas maksimum stack sebesar 1000, variabel top_3021 sebagai penunjuk elemen teratas, serta array a sebagai tempat penyimpanan data stack.

2. stackArray_2511533021() { top_3021 = -1; }
   Konstruktor untuk menginisialisasi stack dalam keadaan kosong, ditandai dengan nilai top_3021 = -1.
3. boolean isEmpty_3021() { return (top_3021 < 0); }
   Method untuk mengecek apakah stack kosong, yaitu ketika nilai top_3021 kurang dari 0.
4. boolean push_3021(int x)
   Method untuk menambahkan elemen ke dalam stack. Jika stack sudah penuh (overflow), maka akan menampilkan pesan error. Jika tidak, elemen akan dimasukkan ke posisi teratas.
5. int pop_3021()
   Method untuk menghapus elemen teratas dari stack. Jika stack kosong (underflow), akan menampilkan pesan error. Jika tidak, elemen teratas diambil lalu top_3021 dikurangi.
6. int peek_3021()
   Method untuk melihat elemen paling atas tanpa menghapusnya. Jika stack kosong, akan menampilkan pesan error.
7. void print_3021()
   Method untuk menampilkan seluruh isi stack dari elemen paling atas ke bawah menggunakan perulangan.

2.1.2 Class stackArrayDriver

```java
package pekan3_2511533021;

public class stackArrayDriver_2511533021 {

    public static void main(String[] args) {
        stackArray_2511533021 s = new stackArray_2511533021 ();
        s.push_3021(10);
        s.push_3021(20);
        s.push_3021(30);
        System.out.println(s.pop_3021() + " dikeluarkan dari stack");
        System.out.println("elemen teratas adalah : " + s.peek_3021());
        System.out.print("element pada stack    :");
        s.print_3021();

    }

}
```

1. stackArray_2511533021 s = new stackArray_2511533021();
   Membuat objek s dari class stack untuk digunakan dalam operasi stack.

2. s.push_3021(10); s.push_3021(20); s.push_3021(30);
   Menambahkan tiga elemen (10, 20, 30) ke dalam stack secara berurutan, di mana elemen terakhir (30) berada di posisi teratas.

3. System.out.println(s.pop_3021() + " dikeluarkan dari stack");
   Menghapus elemen teratas dari stack (30) dan menampilkannya sebagai elemen yang dikeluarkan.

4. System.out.println("elemen teratas adalah : " + s.peek_3021());
   Menampilkan elemen teratas saat ini tanpa menghapusnya, yaitu 20.

5. System.out.print("element pada stack :"); s.print_3021();
   Menampilkan seluruh isi stack dari atas ke bawah, yaitu 20 dan 10.

2.1.3 Class StackPostfix

```java
package pekan3_2511533021;
import java.util.Scanner;
import java.util.Stack;

public class StackPostfix_25115330021 {

    public static int postfixEvaluate_3021(String expression) {
        Stack <Integer> s = new Stack <Integer>();
        Scanner input_3021 = new Scanner (expression);
        while (input_3021.hasNext()) {
            if (input_3021.hasNext()) {
                s.push(input_3021.nextInt());
            } else {
                String operator_3021 = input_3021.next();
                int operand2 = s.pop();
                int operand1 = s.pop();
                if (operator_3021.equals("+")) {
                    s.push (operand1+operand2);
                } else if (operator_3021.equals("-")){
                    s.push(operand1-operand2);
                }else if (operator_3021.equals("*")) {
                    s.push(operand1*operand2);
                }else {
                    s.push (operand1/operand2);
                }
            }
        }
        input_3021.close ();
        return s.pop();
    }
    public static void main (String []args) {
```

```java
    }
    public static void main (String []args) {
        System.out.println("hasil postfix = "+ postfixEvaluate_3021("5 2 5 * + 7 - "));
    }

}
```

1. Stack<Integer> s = new Stack<Integer>();
   Membuat stack bertipe Integer yang digunakan untuk menyimpan operand saat proses perhitungan postfix.

2. Scanner input_3021 = new Scanner(expression);
   Menggunakan Scanner untuk membaca setiap elemen (angka dan operator) dari string ekspresi postfix.

3. while (input_3021.hasNext())
   Perulangan untuk membaca seluruh isi ekspresi satu per satu sampai habis.

4. if (input_3021.hasNextInt()) { s.push(input_3021.nextInt()); }
   Jika token yang dibaca adalah angka, maka langsung dimasukkan (push) ke dalam stack.

5. else { ... }
   Jika token bukan angka (berarti operator), maka dilakukan proses perhitungan.

6. int operand2 = s.pop(); int operand1 = s.pop();
   Mengambil dua nilai teratas dari stack sebagai operand untuk operasi matematika.

7. if (operator_3021.equals("+")) ...
   Melakukan operasi sesuai operator (+, -, *, /), lalu hasilnya dimasukkan kembali ke stack.

8. return s.pop();
   Setelah semua operasi selesai, hasil akhir diambil dari stack dan dikembalikan sebagai output.

9. System.out.println("hasil postfix = " + postfixEvaluate_3021("5 2 5 * + 7 - "));
   Menjalankan fungsi evaluasi postfix dengan contoh ekspresi, lalu menampilkan hasilnya.

2.1.4 Class SiswaStack

```java
package pekan3_2511533021;
import java.util.ArrayList;
class Siswa_2511533021 {
    String nama_3021;
    int nim_3021;

    public Siswa_2511533021 (String nama_3029, int nim_3021) {
        this.nama_3021= nama_3021;
        this.nim_3021= nim_3021;

    }

    @Override
    public String toString () {
        return "NIM: " + nim_3021 + ", Nama : " +nama_3021;
    }
}
public class SiswaStack_2511533021 {
    private ArrayList<Siswa_2511533021> stack_3021;

    public SiswaStack_2511533021() {
        stack_3021 = new ArrayList<>();
    }
    public void push (Siswa_2511533021 mhs) {
        stack_3021.add(mhs);
    }

public Siswa_2511533021 pop() {
    if (!isEmpty()) {
        return stack_3021.remove(stack_3021.size() -1);
     }
```

```java
    }
    return null;
}

public boolean isEmpty () {
    return stack_3021.isEmpty();
    }
 public void tampilkanSiswa_3021() {
    for (int i= stack_3021.size() -1; i>=0; i--) {
        System.out.println(stack_3021.get(i));
    }
 }
 public static void main (String [] args) {
    SiswaStack_2511533021 studentstack = new SiswaStack_2511533021();

    Siswa_2511533021 mhs1 = new Siswa_2511533021 ("Ali ", 1);
    Siswa_2511533021 mhs2 = new Siswa_2511533021 ("Boby ", 2);
    Siswa_2511533021 mhs3 = new Siswa_2511533021 ("Charles ", 3);

    studentstack.push(mhs1);
    studentstack.push(mhs2);
    studentstack.push(mhs3);

    System.out.println("siswa di dalam stack : ");
    studentstack.tampilkanSiswa_3021();

    System.out.println("siswa teratas " + studentstack.peek());
    System.out.println("mengeluarkan siswa teratas dari stack " + studentstack.pop());
    System.out.println("daftar siswa setelah di pop : ");
    studentstack.tampilkanSiswa_3021();
 }
```

1. class Siswa_2511533021 { String nama_3021; int nim_3021; ... }
   Membuat class Siswa sebagai ADT yang menyimpan data mahasiswa berupa nama dan NIM.

2. public Siswa_2511533021(String nama_3021, int nim_3021)
   Konstruktor untuk menginisialisasi objek siswa saat dibuat. *(Catatan: di kode kamu ada typo, seharusnya parameter nama_3021 dipakai di this.nama_3021 = nama_3021;)*

3. public String toString()
   Method untuk menampilkan data siswa dalam bentuk string agar mudah dicetak.

4. private ArrayList<Siswa_2511533021> stack_3021;
   Mendeklarasikan ArrayList sebagai struktur stack untuk menyimpan objek siswa.

5. public SiswaStack_2511533021()
   Konstruktor untuk menginisialisasi stack sebagai ArrayList kosong.

6. public void push(Siswa_2511533021 mhs)
   Method untuk menambahkan siswa ke dalam stack (dimasukkan ke posisi terakhir/top).

7. public Siswa_2511533021 pop()
   Method untuk menghapus dan mengembalikan elemen teratas dari stack (prinsip LIFO).

8. public Siswa_2511533021 peek()
   Method untuk melihat elemen teratas tanpa menghapusnya.

9. public boolean isEmpty()
   Method untuk mengecek apakah stack kosong atau tidak.

10. public void tampilkanSiswa_3021()
    Method untuk menampilkan seluruh isi stack dari atas ke bawah menggunakan perulangan.

11. SiswaStack_2511533021 studentstack = new SiswaStack_2511533021();
    Membuat objek stack untuk menyimpan data siswa.

12. Siswa_2511533021 mhs1 = ...
    Membuat beberapa objek siswa sebagai data yang akan dimasukkan ke stack.

13. studentstack.push(mhs1);
    Menambahkan objek siswa ke dalam stack.

14. studentstack.tampilkanSiswa_3021();
    Menampilkan seluruh data siswa dalam stack.

15. studentstack.peek()
    Menampilkan siswa yang berada di posisi teratas.

16. studentstack.pop()
    Menghapus siswa yang berada di posisi teratas.

17. studentstack.tampilkanSiswa_3021();
    Menampilkan kembali isi stack setelah dilakukan operasi pop.

2.1.5 Class NilaiMaksimum

```java
package pekan3_2511533021;
import java.util.Stack;
public class NilaiMaksimum_2511533021 {
    public static int max (Stack <Integer>s_3021) {
    Stack <Integer> backup_3021 = new Stack <Integer>();
    int maxValue_3021 = s_3021.pop();
    backup_3021.push (maxValue_3021);
    while (!s_3021.isEmpty()) {
        int next_3021=s_3021.pop();
        backup_3021.push(next_3021);
        maxValue_3021 = Math.max(maxValue_3021, next_3021);
    }
    while (!backup_3021.isEmpty() ) {
        s_3021.push(backup_3021.pop());
    }
    return maxValue_3021;
  }
    public static void main(String[] args) {
        Stack <Integer> s_3021 = new Stack<Integer> ();
        s_3021.push (70);
        s_3021.push (12);
        s_3021.push (20);
        System.out.println("isi stack " + s_3021);
        System.out.println("Stack Teratas " + s_3021.peek());
        System.out.println("nilai maksimum " + max(s_3021));


    }

}

```

1. public static int max(Stack<Integer> s_3021)
   Method ini digunakan untuk mencari nilai maksimum dalam stack tanpa menghilangkan isi stack secara permanen.

2. Stack<Integer> backup_3021 = new Stack<Integer>(); dan int maxValue_3021 = s_3021.pop();
   Membuat stack cadangan untuk menyimpan sementara data, lalu mengambil elemen pertama sebagai nilai awal maksimum.

3. while (!s_3021.isEmpty()) { ... }
   Mengambil semua elemen dari stack utama satu per satu, menyimpannya ke stack cadangan, sekaligus membandingkan untuk mendapatkan nilai maksimum menggunakan Math.max().

4. while (!backup_3021.isEmpty()) { s_3021.push(backup_3021.pop()); }
   Mengembalikan semua elemen dari stack cadangan ke stack utama agar data tetap seperti semula.

5. return maxValue_3021;
   Mengembalikan nilai maksimum yang telah ditemukan.

6. main() (pengujian program)
   Membuat stack, menambahkan beberapa nilai (70, 12, 20), menampilkan isi stack, melihat elemen teratas, lalu memanggil method max() untuk menampilkan nilai maksimum.

BAB III
PENDAHULUAN

3.1 Kesimpulan

Berdasarkan pembelajaran mengenai struktur data Stack, dapat disimpulkan bahwa Stack merupakan struktur data yang bekerja dengan prinsip *LIFO (Last In First Out)*, di mana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar. Stack dapat diimplementasikan menggunakan array maupun ArrayList, serta menyediakan operasi dasar seperti *push*, *pop*, dan *peek*. Penerapan Stack sangat berguna dalam berbagai kasus seperti evaluasi ekspresi, manajemen memori, serta pengolahan data secara berurutan. Dengan memahami konsep ini, mahasiswa dapat mengelola data secara lebih terstruktur dan efisien dalam pemrograman.

3.2 Saran

Dalam mempelajari Stack, disarankan untuk lebih sering berlatih dengan berbagai contoh program agar pemahaman semakin mendalam. Selain itu, penting untuk memahami konsep dasar sebelum mengimplementasikannya ke dalam kode program. Mahasiswa juga disarankan untuk mencoba berbagai variasi implementasi Stack, seperti menggunakan array, ArrayList, maupun class bawaan Java, serta mengkombinasikannya dengan algoritma lain agar kemampuan pemrograman semakin berkembang.

DAFTAR PUSTAKA

GeeksforGeeks. (2025). *Stack Class in Java*. Diakses dari: https://www.geeksforgeeks.org/stack-class-in-java/ ()